DETAILED TECHNICAL STUDY NOTES

ReDiffDet

Rotation-Equivariant Diffusion Model for Oriented Object Detection

Based on the CVPR 2025 paper

Training, inference, experimental results,
mathematical interpretation, and critical evaluation

Part 3

Training, inference, comparison with DiffusionDet, experiments, and ablations

Contents

Part 3 — Training, inference, DiffusionDet comparison, experiments, and ablations **1**

9. Training ReDiffDet: the complete algorithm . . . 1
   - 9.1 Training inputs . . . 1
   - 9.2 Step-by-step training . . . 1
   - 9.3 What is the purpose of randomizing the timestep? . . . 2
   - 9.4 The loss function . . . 2

10. Inference: detecting objects from noisy Gaussian states . . . 2
   - 10.1 Step-by-step inference . . . 3
   - 10.2 What is DDIM, and why does ReDiffDet use it? . . . 3
   - 10.3 What is box renewal? . . . 3
   - 10.4 Why does the model use multiple iterations? . . . 4

11. The three main ideas compared with DiffusionDet . . . 4
   - 11.1 Why not simply add noise to the angle? . . . 4
   - 11.2 Why not use a Gaussian distribution for every object in the image directly? 5

12. Experimental setup . . . 5
   - 12.1 What is AP50? . . . 6

13. Main experimental results . . . 7
   - 13.1 Results on DOTA-v1.0 . . . 7
   - 13.2 Results on DOTA-v1.5 and DOTA-v2.0 . . . 7
   - 13.3 Results on DIOR-R small objects . . . 8
   - 13.4 Results on COCO . . . 8

14. Ablation studies: which components matter? . . . 9
   - 14.1 Number of decoder layers . . . 9
   - 14.2 Scaling factor (m) . . . 9
   - 14.3 Number of sampled points . . . 10
   - 14.4 Signal scale . . . 10
   - 14.5 Incrementally adding the proposed components . . . 10
   - 14.6 Number of inference steps . . . 10

Part 3 — Training, inference, DiffusionDet comparison, experiments, and ablations

9. Training ReDiffDet: the complete algorithm

Let's walk through Algorithm 1 on page 5, connecting every operation to the theory.

9.1 Training inputs

The algorithm receives:

- An image.
- Ground-truth boxes.
- Ground-truth labels.
- The diffusion horizon (T).

The goal is to minimize a detection loss that makes the decoder predict the correct objects from noisy Gaussian states.

9.2 Step-by-step training

Step 1 — Extract image features

$$F = \text{Backbone} + \text{FPN}(I)$$

The network builds the multiscale visual representation.

Step 2 — Generate conditional boxes

$$B_{\{\text{cond}\}} = E_{\{\text{cond}\}}(F)$$

The encoder proposes and scores candidate boxes.

Step 3 — Convert boxes into Gaussian distributions

Ground-truth boxes become target Gaussians. Conditional boxes are also represented as Gaussians, and the target set is padded to the required number of instances.

Step 4 — Sample and corrupt

Sample points from the target distributions and apply the forward diffusion equation at a randomly selected timestep.

Step 5 — Reconstruct the noisy Gaussian

Use MLE to estimate a Gaussian from the corrupted points. This becomes the decoder's noisy input.

Step 6 — Predict boxes and classes

The aligned decoder uses the noisy Gaussian state, image features and timestep to predict object boxes and labels.

Step 7 — Calculate detection loss

Compare predicted boxes and classes with ground truth using set-prediction supervision, then update the model parameters through backpropagation.

9.3 What is the purpose of randomizing the timestep?

During training, the timestep is sampled randomly:

$$t \sim \text{Uniform}(\{0, 1, \ldots, T\}).$$

The model therefore sees examples at different noise levels.

If it were trained only on lightly corrupted states, it might fail to recover objects from heavily corrupted candidates. If trained only on highly corrupted states, it might not learn to refine nearly correct boxes effectively.

Random timestep training teaches the decoder to handle different stages of the denoising process.

9.4 The loss function

The paper states that its predictions are supervised using a set-prediction loss.

Conceptually,

$$\mathcal{L}_{\{\text{det}\}} = \mathcal{L}_{\{\text{cls}\}} + \lambda_{\{\text{box}\}}\mathcal{L}_{\{\text{box}\}} + \lambda_{\{\text{geom}\}}\mathcal{L}_{\{\text{geom}\}},$$

where the terms stand for classification, box regression and any geometric regression components in the detector's implementation.

This expression is a generic way to describe a detection loss, not an assertion that the paper introduces exactly this three-term formula with these names and weights. The precise loss construction is inherited from the DiffusionDet/MMRotate implementation and the oriented-box configuration.

The important distinction is that the network is trained using detection supervision on predicted boxes and labels, not solely by minimizing a Gaussian noise-prediction loss.

The Gaussian representation provides the state space for diffusion; the detector's supervised objective teaches the model to predict actual objects.

10. Inference: detecting objects from noisy Gaussian states

Training teaches the model how to recover boxes from noisy states. Inference applies the learned reverse process to new images.

Algorithm 2 on page 5 describes the following procedure.

10.1 Step-by-step inference

1. Extract image features using the backbone and FPN.
2. Generate conditional boxes from the image features.
3. Transform standard Gaussian samples according to the conditional boxes to obtain the initial Gaussian states.
4. Choose a sequence of reverse diffusion timesteps.
5. At each timestep, run the aligned decoder to predict boxes and labels.
6. Convert the predicted boxes back into Gaussian distributions.
7. Use a DDIM update to move to the next denoising state.
8. Apply the box-renewal strategy to revive high-scoring predictions.
9. Return the final object predictions.

The most important difference between training and inference is the source of the noisy state.

- During training, the model creates noisy states from ground-truth Gaussian representations.
- During inference, the model starts from transformed standard-normal samples conditioned on image-derived candidate boxes.

10.2 What is DDIM, and why does ReDiffDet use it?

DDIM stands for Denoising Diffusion Implicit Models.

A conventional diffusion sampler may take many small reverse steps. DDIM permits a sampling schedule with fewer steps while retaining the model's learned denoising predictions.

In ReDiffDet, DDIM is used to update the Gaussian state from the current timestep to the next selected timestep.

The paper does not require the inference sequence to visit every possible timestep. Instead, the selected timesteps can form a shorter schedule.

This matters because object detection must be computationally practical. Each reverse step requires the decoder to process candidate instances and their image features.

10.3 What is box renewal?

The paper adopts the box-renewal strategy from DiffusionDet.

The idea is to prevent promising object predictions from being lost during iterative refinement. High-scoring predictions can be retained or used to revive candidate boxes, helping the process maintain useful object hypotheses.

The exact renewal logic is an implementation detail inherited from the baseline. Its conceptual role is to keep useful candidates available as the denoising trajectory progresses.

10.4 Why does the model use multiple iterations?

A single reverse step can already produce predictions, but further refinement may improve localization.

The paper investigates iterative evaluation with up to eight steps. It reports that accuracy increases as the number of iterative steps grows, with diminishing returns at higher step counts.

However, more steps require more computation. The paper's default oriented-detection evaluation uses 300 boxes and one iteration, while its analysis explores additional steps and different numbers of dynamic boxes.

This is a practical trade-off between detection quality and inference cost.

11. The three main ideas compared with DiffusionDet

The most useful way to understand ReDiffDet's contribution is to compare it with its baseline, DiffusionDet.

DiffusionDet already treats object detection as a generative denoising task. ReDiffDet adapts and extends that idea for oriented boxes.

Component	DiffusionDet	ReDiffDet
Main task	Horizontal object detection	Oriented object detection
Box representation	Box coordinates, including positions and sizes	2D Gaussian distribution derived from an oriented box
Rotation equivariance	Not explicitly established by the same framework	Explicitly formulated and theoretically analyzed
Candidate initialization	Random boxes	Image-conditioned boxes and Gaussian states
Region-feature alignment	Does not use the proposed oriented alignment strategy	Uses RiRoI Align
Iterative refinement	Denoising of candidate boxes	Denoising of Gaussian box representations

The paper's claim is not that diffusion itself is new. The contribution is to make diffusion more appropriate for oriented object detection by changing the representation, introducing conditional initialization and aligning the decoder with rotated geometry.

11.1 Why not simply add noise to the angle?

One might ask why we need a Gaussian distribution at all. Why not represent a box as five numbers and add noise to each coordinate?

The difficulty is that these coordinates do not have identical mathematical structure.

- The center coordinates are ordinary spatial coordinates.
- Width and height are positive quantities.

- Angles are periodic.
- Width and height may exchange roles under equivalent rectangle representations.
- A small numerical change in the angle can have a different geometric effect depending on the box's aspect ratio.

For example, a change of 10° in the angle of a long, thin ship may move its corners substantially, while the same angle change for a nearly square building may have much less visible impact.

A 2D Gaussian instead encodes the center, orientation and spread through a mean and covariance. The covariance transforms naturally under rotations.

This is the mathematical advantage of the representation. It gives the diffusion process a geometrically meaningful object to operate on.

11.2 Why not use a Gaussian distribution for every object in the image directly?

A Gaussian is a continuous probability distribution, whereas an object detector predicts a finite set of instances.

ReDiffDet bridges that difference by:

1. Representing each box as a Gaussian.
2. Sampling a finite number of points from the distribution.
3. Estimating the distribution from those points.
4. Converting the distribution back to an oriented box.

This gives the model a representation that is compatible with geometric transformations, while preserving the output format expected by an object detector.

The finite sampling step is not free: a small number of samples introduces estimation variability. The paper investigates this trade-off experimentally.

12. Experimental setup

The paper evaluates ReDiffDet on five datasets or dataset variants.

Dataset	Purpose and characteristics
DIOR-R	Large-scale oriented object detection; 23,463 images and 20 classes
DOTA-v1.0	Aerial oriented detection; 2,806 images and 15 categories
DOTA-v1.5	Extended DOTA version with 16 categories
DOTA-v2.0	Larger and more challenging DOTA version with 18 categories and 11,268 images

Dataset	Purpose and characteristics
COCO	Horizontal object detection; used to test the conditional strategy beyond oriented boxes

The paper's implementation details include:

- Two NVIDIA RTX 2080 Ti GPUs.
- Batch size 4.
- AdamW optimizer.
- Weight decay $(10^{\{} - 4\})$.
- Initial learning rate $(4{\times}10^{\{} - 5\}) for DIOR - R$.
- Initial learning rate $(5.5{\times}10^{\{} - 5\}) for the DOTA datasets$.
- A 24-epoch training schedule, with learning-rate reductions at epochs 16 and 22.
- Random flipping as the stated data augmentation strategy.
- $(1024{\times}1024) crops with 200 - pixel overlap for DOTA training$.

For COCO, the authors follow the DiffusionDet setup with a 12-epoch training schedule.

These are the paper's reported settings, not universal hyperparameters for all ReDiffDet implementations. See Section 4.1, page 5.

12.1 What is AP50?

The main reported metric is AP50, or average precision at an intersection-over-union threshold of 0.50.

For a predicted box $(\mathrm{B}_p) and ground - truth box(B_g)$,

$$\mathrm{IoU}(B_p, B_g) = \frac{|B_p \cap B_g|}{|B_p \cup B_g|}.$$

For oriented detection, the intersection and union are calculated using the rotated boxes.

At IoU threshold 0.50, a prediction can count as a true positive when its overlap with a matched ground-truth object is sufficiently high and its category is correct, subject to the evaluation's matching protocol.

Precision and recall are

$$\mathrm{Precision} = \frac{TP}{TP + FP},$$

$$\mathrm{Recall} = \frac{TP}{TP + FN}.$$

Average precision summarizes the precision-recall curve, and mean average precision averages the AP values across categories.

AP50 is useful, but it does not tell us everything about localization quality. AP75 imposes a stricter overlap threshold, while COCO-style AP50:95 averages across multiple IoU thresholds.

13. Main experimental results

13.1 Results on DOTA-v1.0

The paper's Table 1 reports the following AP50 results:

Method/configuration	Backbone	AP50
DiffusionDet-O	ResNet-50	60.50%
ReDiffDet	ReResNet-50	75.45%
ReDiffDet	ResNet-50	76.18%
ReDiffDet	Swin-T	76.62%
ReDiffDet, multi-scale	ReResNet-50	80.70%

The comparison with DiffusionDet-O is especially revealing. The baseline obtains 60.50% AP50, whereas ReDiffDet with ReResNet-50 obtains 75.45%.

The absolute improvement is

$$75.45 - 60.50 = \boxed{14.95 \text{ percentage points}}.$$

This is a substantial difference in the reported results.

However, we must interpret it carefully: this comparison changes multiple aspects of the system, including the oriented-box formulation, candidate conditioning and decoder alignment. It should not be interpreted as proof that the Gaussian representation alone accounts for the entire gain.

The paper also reports 80.70% AP50 with multi-scale training and testing. That setting is different from the standard single-scale configuration and should be compared with other methods using compatible evaluation settings.

13.2 Results on DOTA-v1.5 and DOTA-v2.0

Table 2 reports:

Dataset	ReDiffDet AP50	Comparison highlighted by the paper
DOTA-v1.5	68.00%	DCFL: 67.37%
DOTA-v2.0	55.76%	OrientedFormer: 54.27%

These results suggest that ReDiffDet's benefits are not confined to DOTA-v1.0.

The datasets differ in annotation and category complexity, so the absolute AP50 values should not be compared directly across datasets. The relevant comparison is between methods evaluated on the same dataset under compatible protocols.

13.3 Results on DIOR-R small objects

Table 3 examines small-object performance on DIOR-R:

ReDiffDet backbone	AP50
PKINet-T	65.47%
LSK-T	66.62%
ReResNet-50	68.05%

The paper emphasizes improvements for small objects and objects with large aspect ratios.

This makes sense in the context of the architecture:

- Gaussian geometry encodes orientation and spread.
- Conditional boxes provide candidate locations.
- Rotated-region alignment helps isolate the object.
- Iterative refinement can improve localization.

Nevertheless, these mechanisms are a plausible explanation of the observed results, not a controlled demonstration that each one independently explains all small-object improvements.

13.4 Results on COCO

The paper also evaluates its conditional strategy on horizontal object detection using COCO.

Method	mAP	AP50	AP75
DiffusionDet	43.2	62.6	46.1
DiffusionDet + condition	45.0	64.6	48.4
Absolute improvement	+1.8	+2.0	+2.3

The conditional strategy improves all three reported metrics.

This is an important result because it suggests that image-conditioned candidate generation can benefit diffusion-based detection even when oriented boxes are not the main task.

It does not, however, demonstrate that the entire rotation-equivariant Gaussian framework is necessary for horizontal detection. The experiment specifically evaluates the conditioning strategy in the DiffusionDet setup.

14. Ablation studies: which components matter?

An ablation study removes or changes individual design choices to understand their effect on performance.

The paper's Table 5 and Figures 4–5 are particularly useful because they test several decisions behind ReDiffDet.

14.1 Number of decoder layers

The authors compare three to seven decoder layers.

Decoder layers	Parameters	AP50	AP75
3	51M	67.25	49.14
4	64M	68.05	49.96
5	78M	67.74	49.65
6	92M	66.83	48.72
7	106M	66.25	48.16

Four layers perform best among the tested configurations.

Why might additional layers hurt?

More layers increase model capacity, but they also increase parameters and computation. Deeper refinement can be less effective if the additional layers do not improve the representation enough to justify their cost.

The experiment demonstrates that adding decoder depth is not guaranteed to improve detection accuracy.

14.2 Scaling factor (m)

The Gaussian covariance contains

$$\Lambda = \operatorname{diag}\left((w/m)\left(\frac{w}{m}\right)^2, \left(\frac{h}{m}\right)^2\right).$$

The scaling factor controls the spread of the sampled Gaussian points.

The paper finds that $m = 6$ works best in its ablation experiment.

A small (m) produces a wider Gaussian, while a large (m) produces a narrower one. Both extremes can make the sampled representation less robust to the diffusion noise.

The best value therefore balances the scale of the geometric signal against the noise added during training.

This is an empirical design choice rather than a universal mathematical constant.

14.3 Number of sampled points

The authors test different sample counts (k) and find that $k = 8$ performs best among the evaluated settings.

The reason is a trade-off:

- Too few samples make the estimated mean and covariance highly sensitive to individual noise perturbations.
- More samples can improve the stability of estimation.
- But additional samples do not necessarily improve the detector's downstream performance.

The reported optimum is therefore specific to the paper's configuration and training procedure.

14.4 Signal scale

The paper varies the signal scale and finds that a scale of 2.0 works best in the tested configuration.

Its explanation is geometric: the sampled points should have a scale compatible with the range of the added Gaussian noise.

If the signal is too weak, noise can overwhelm it. If it is too strong, the corrupted samples may deviate from the distribution the decoder has learned to handle.

This is another example of how the statistical representation and the diffusion schedule interact.

14.5 Incrementally adding the proposed components

Table 5e gives a particularly useful comparison:

Configuration	AP50
DiffusionDet-O baseline	49.91%
Add rotation-equivariant diffusion formulation	52.15%
Also add conditional encoder	60.02%
Also add aligned decoder	62.17%
Full configuration in the reported ablation	68.05%

The table shows progressive improvements as the proposed components are introduced.

One must read the sequence according to the exact configurations in the original table. It is evidence that the combination of components is useful in the authors' experimental setup, not a general theorem that each module must improve every possible detector.

14.6 Number of inference steps

Figure 4 investigates iterative evaluation and dynamic box counts.

The authors find that increasing the number of denoising iterations improves accuracy, with diminishing gains as the number of steps increases.

This reflects a general refinement trade-off: more steps give the model more opportunities to adjust candidate geometry, but each step incurs additional computation.

The number of candidate boxes also matters. Too few candidates may not cover all objects, while too many create more work and potentially more false positives.

The paper's default setting of 300 boxes and one iteration is therefore a practical operating point, not a claim that it is optimal for every dataset or hardware configuration.